Linux 应用开发手册

Revision History

Draft Date	Revision No.	Description
2024/06/25	V1.0	1. 初始版本。

目　　录

前　言 4

1　Linux 常用开发案例 5

1.1　led_flash 案例 5

1.1.1　案例功能 5

1.1.2　操作说明 5

1.1.3　关键代码 6

1.2　key_demo 案例 7

1.2.1　案例功能 7

1.2.2　操作说明 8

1.2.3　关键代码 8

1.3　uart_rw 案例 9

1.3.1　案例功能 9

1.3.2　操作说明 10

1.3.3　关键代码 15

1.4　lcd_demo 案例 19

1.4.1　案例功能 19

1.4.2　操作说明 20

1.4.3　关键代码 24

1.5　tcp_udp 案例 27

1.5.1　案例功能 27

1.5.2　操作说明 29

1.5.3　关键代码 32

1.6　can_echo 案例 32

1.6.1　案例功能 32

1.6.2　操作说明 32

1.6.3　关键代码 38

2　Python 开发案例 38

2.1 led_flash 案例 40
2.1.1 案例功能 40
2.1.2 操作说明 41
2.1.3 关键代码 41
2.2 key_demo 案例 42
2.2.1 案例功能 42
2.2.2 操作说明 43
2.2.3 关键代码 44
更多帮助 46

前　言

本文档涉及的开发案例位于产品资料“4-软件资料\Demo\”路径下的 base-demos 目录和 python-demos 目录。

base-demos 目录存放 Linux 常用开发案例，案例 bin 目录存放可执行文件，案例 src 目录存放源码。python-demos 目录存放 Python 开发案例，案例脚本文件无需编译，可基于我司提供的文件系统直接运行。

如需重新编译 Linux 常用开发案例，请将对应案例拷贝至 Ubuntu 工作目录下，进入 src 目录执行如下命令加载 Linux SDK 环境变量，并执行 make 命令进行案例编译。编译完成后，将在当前目录下生成可执行文件。

Host#　cd base-demos/led_flash/src/

Host#　source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host#　make

```
tronlong@tronlong-virtual-machine:~$ cd base-demos/led_flash/src/
tronlong@tronlong-virtual-machine:~/base-demos/led_flash/src$ pwd
/home/tronlong/base-demos/led_flash/src
tronlong@tronlong-virtual-machine:~/base-demos/led_flash/src$ ls
led_flash.c  makefile
tronlong@tronlong-virtual-machine:~/base-demos/led_flash/src$ source /home/tron
long/SDK/environment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/base-demos/led_flash/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2
 -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysro
ot=/home/tronlong/SDK/sysroots/armv8a-poky-linux -Wall led_flash.c -o led_flash
tronlong@tronlong-virtual-machine:~/base-demos/led_flash/src$ ls
led_flash  led_flash.c  makefile
tronlong@tronlong-virtual-machine:~/base-demos/led_flash/src$
```

图 1

1 Linux 常用开发案例

1.1 led_flash 案例

1.1.1 案例功能

本案例通过向评估底板用户可编程指示灯设备节点反复交替写入 1、0 数值，实现 LED 闪烁效果。LED 点亮与熄灭时间均为 0.5s。

程序流程如下图所示：

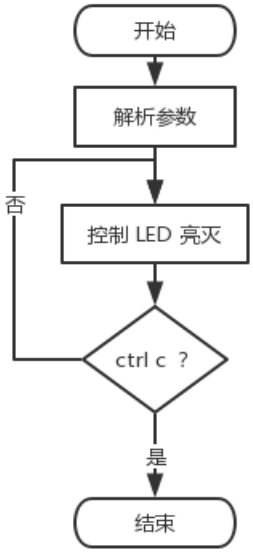

图 2

LED 设备节点为"/sys/class/leds/user-ledX/"目录下的 brightness。

```
root@IMX8MP-Tronlong:~# ls /sys/class/leds/
disk  heartbeat  mmc1::  mmc2::  user-led0
root@IMX8MP-Tronlong:~# ls /sys/class/leds/user-led0
brightness  device  max_brightness  power  subsystem  trigger  uevent
root@IMX8MP-Tronlong:~#
```

图 3

1.1.2 操作说明

将本案例 bin 目录下的可执行程序 led_flash 拷贝至评估板文件系统，并在可执行程序所在目录执行如下命令运行程序，即可看到评估底板 LED1 以 0.5s 的时间间隔进行闪

烁。同时，串口终端打印系统全部 LED 设备信息和程序当前控制的 LED 设备信息。

Target# ./led_flash -h

Target# ./led_flash -n 1

```
root@IMX8MP-Tronlong:~# ls led_flash
led_flash
root@IMX8MP-Tronlong:~# ./led_flash -h
Usage: led_flash [options]

Options:
 -n | --number        Number of LEDs, range of 1 to 4
 -v | --version       Display version information
 -h | --help          Show help content

root@IMX8MP-Tronlong:~# ./led_flash -n 1

System leds :
/sys/class/leds/disk
/sys/class/leds/heartbeat
/sys/class/leds/mmc1::
/sys/class/leds/mmc2::
/sys/class/leds/user-led0

Flashing leds :
/sys/class/leds/user-led0
```

图 4

1.1.3 关键代码

（1） 预定义 LED 数组。程序由此数组获取 LED 信息，数组信息必须为系统已有 LED 信息，否则程序运行报错。

```
/* User-operable LEDs */
static char *g_leds[] = {
    "/sys/class/leds/user-led0",
    "/sys/class/leds/user-led1",
    "/sys/class/leds/user-led2",
    "/sys/class/leds/user-led3"
};
```

图 5

（2） LED 亮灭操作和时间间隔。

```
131     while (!g_quit) {
132         /* Turn on LEDs */
133         for (i = 0; i < num; i++) {
134             /* Set the LED brightness value to 1 to turn on the led */
135             snprintf(cmd, 64, "echo 1 > %s/brightness", g_leds[i]);
136             if (system(cmd) != 0) {
137                 fprintf(stderr, "Error: Failed to turn on %s\n", g_leds[i]);
138                 exit(EXIT_FAILURE);
139             }
140         }
141
142         /* Keep the LEDs on for 500 ms */
143         usleep(500 * 1000);
144
145         /* Turn off LEDs */
146         for (i = 0; i < num; i++) {
147             /* Set the LED brightness value to 0 to turn off the LED */
148             snprintf(cmd, 64, "echo 0 > %s/brightness", g_leds[i]);
149             if (system(cmd) != 0) {
150                 fprintf(stderr, "Error: Failed to turn off %s\n", g_leds[i]);
151                 exit(EXIT_FAILURE);
152             }
153         }
154
155         /* Keep the LEDs off for 500 ms */
156         usleep(500 * 1000);
157     }
```

图 6

1.2 key_demo 案例

1.2.1 案例功能

本案例通过监听用户输入按键设备节点状态，检测按键事件。

程序流程如下图所示：

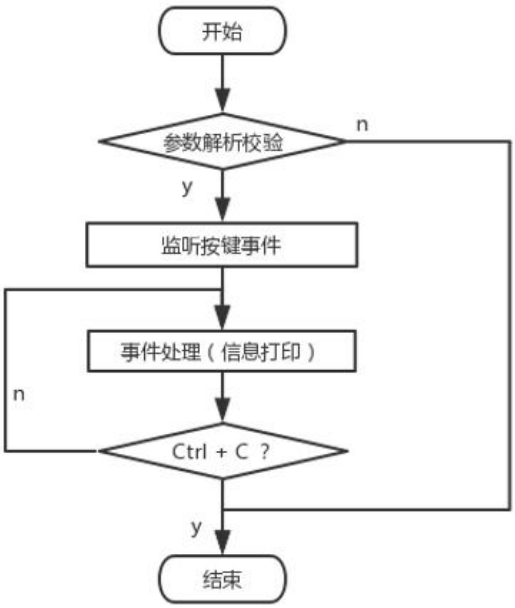

图 7

用户输入按键设备节点为"/dev/input/event0"。获取按键事件后进行按键键值匹配，再进行事件处理。

1.2.2 操作说明

将本案例 bin 目录下的可执行程序 key_demo 拷贝至评估板文件系统，在可执行程序所在目录执行如下命令运行程序，串口终端将打印提示信息。再按下评估板用户输入按键 KEY3，程序将检测到按键事件，并打印按键状态信息。

Target# ./key_demo -h

Target# ./key_demo -d /dev/input/event0

```
root@IMX8MP-Tronlong:~# ./key_demo -h
Usage: key_demo [options]

Options:
 -d | --device        Device
 -v | --version       Display version information
 -h | --help          Show help content

root@IMX8MP-Tronlong:~# ./key_demo -d /dev/input/event0
Please press the key to test.
Key user0 pressed!
Key user0 pressed!
Key user0 pressed!
```

图 8

1.2.3 关键代码

（1） 定义按键。

```
typedef enum { KEY_CODE_NONE = 0, KEY_CODE_USER0, KEY_CODE_USER1 } KeyCode;
```

图 9

（2） 监听按键事件。

```c
static int check_button_pressed(int fd) {
    assert(fd >= 0);

    /* wait button being pressed or released. */
    fd_set input;
    FD_ZERO(&input);
    FD_SET(fd, &input);
    int ret = select(fd + 1, &input, NULL, NULL, NULL);
    if (ret < 0) {
        printf("%s", strerror(errno));
        return -1;
    }

    /* read event */
    struct input_event buf;
    if (read(fd, &buf, sizeof(struct input_event)) < 0) {
        printf("%s", strerror(errno));
        return -1;
    }

    /* Check the input_event value */
    switch (buf.code) {
    case KEY_PROG1:
        /* 1: pressed; 0: released */
        if (buf.value == 1)
            return KEY_CODE_USER0;
        break;
    case KEY_PROG2:
        if (buf.value == 1)
            return KEY_CODE_USER1;
        break;
    default:
        return KEY_CODE_NONE;
        break;
    }

    return KEY_CODE_NONE;
}
```

图 10

（3） 循环监听。

```c
    while (!g_quit) {
        int key_code = check_button_pressed(fd);
        if (key_code < 0)
            continue;

        switch (key_code) {
            case KEY_CODE_USER0:
                printf("Key user0 pressed!\n");
                break;
            case KEY_CODE_USER1:
                printf("Key user1 pressed!\n");
                break;
            default:
                break;
        }
    }
```

图 11

1.3 uart_rw 案例

1.3.1 案例功能

本案例主要实现串口读写功能。

程序流程如下图所示：

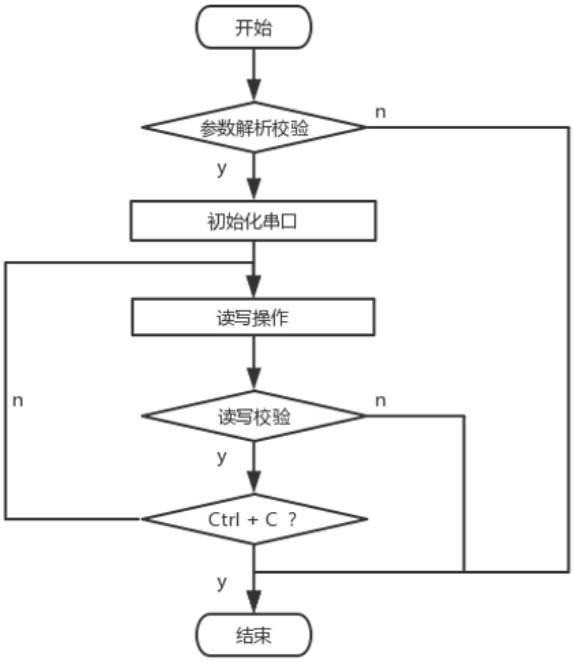

图 12

RS232 UART4 串口设备节点为"/dev/ttymxc3"。RS485 UART1、RS485 UART3 串口设备节点分别为"/dev/ttymxc0"、"/dev/ttymxc2"。UART1、UART3 串口设备节点分别为"/dev/ttymxc0"、"/dev/ttymxc2"。串口初始化时将设置波特率、模式、数据位、停止位等参数。通过设备文件描述符对串口进行读写操作。

1.3.2 操作说明

本小节以 RS232 UART4 串口为例，请使用 RS232 交叉串口母母线、USB 转 RS232 公头串口线，将评估板的 RS232 UART4 串口连接至 PC 机的 USB 接口，如下图所示。

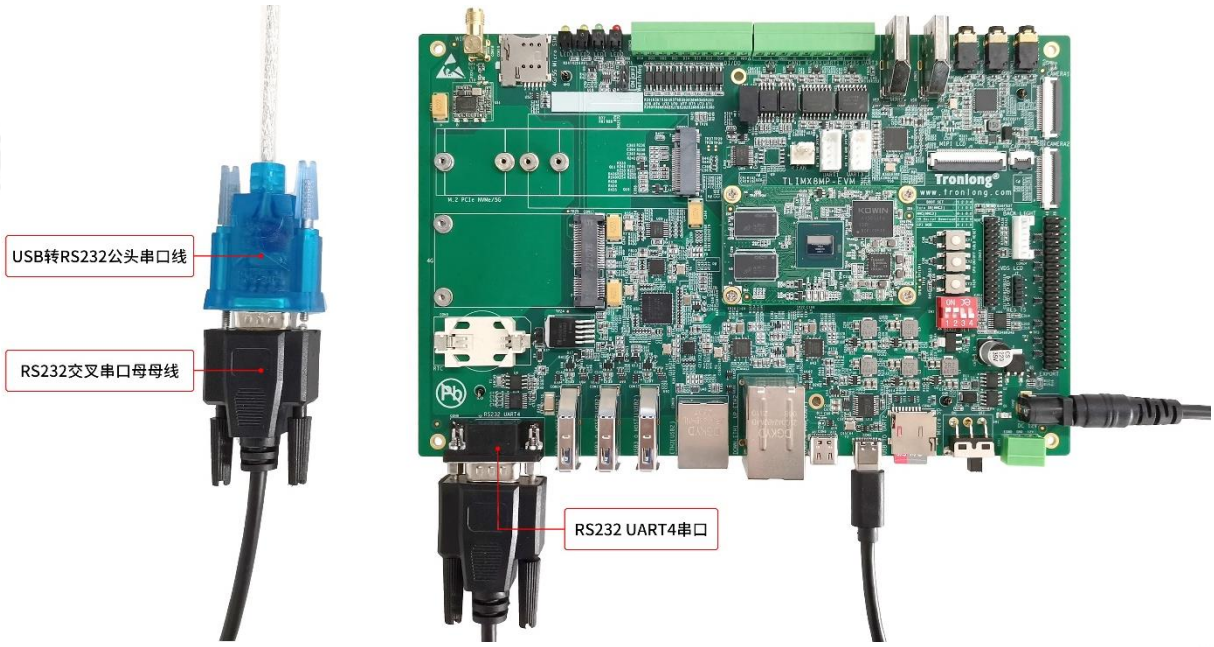

图 13

如需测试 RS485 UART1 或 RS485 UART3 串口，请使用 RS232 转 RS485 模块、USB 转 RS232 公头串口线，将评估板的 RS485 UART1 或 RS485 UART3 串口连接至 PC 机的 USB 接口，如下图所示。

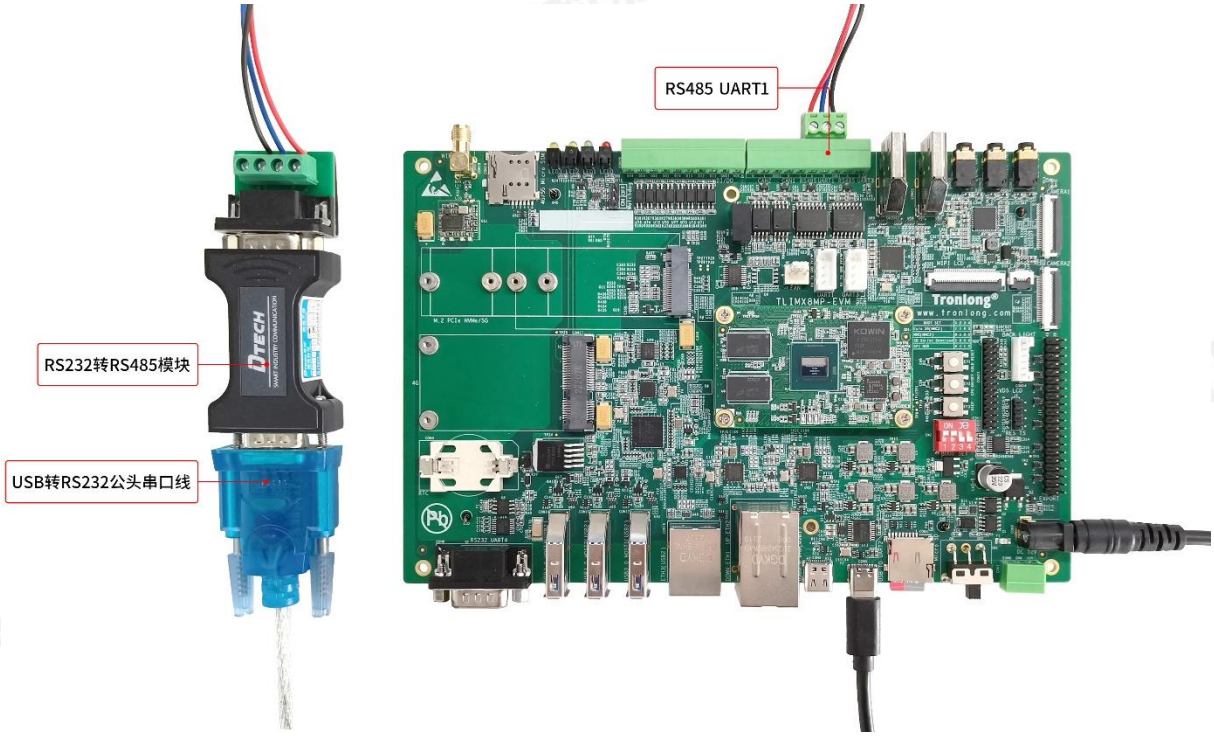

图 14

RS232 转 RS485 模块与评估板上 RS485 UART1 或 RS485 UART3 串口连接方法如下：

（1） RS232 转 RS485 模块 485+端子，连接评估板 RS485 UART1 或 RS485 UART3 串口 A 端子。

（2） RS232 转 RS485 模块 485-端子，连接评估板 RS485 UART1 或 RS485 UART3 串口 B 端子。

（3） RS232 转 RS485 模块 GND 端子，连接评估板 RS485 UART1 或 RS485 UART3 串口 GNDI 端子。

如需测试 UART1 或 UART3 串口，请使用 TTL 转 USB 模块，将评估板的 UART1(CON7)或 UART3(CON9)串口连接至 PC 机的 USB 接口，硬件连接方法如下表所示。

表 1

UART1(CON7)或 UART3(CON9)串口	TTL 转 USB 模块
TX	RXD
RX	TXD
GND	GND

将本案例 bin 目录下的可执行程序 uart_rw 拷贝至评估板文件系统，进入可执行程序所在目录，执行如下命令查看程序参数信息。

Target# ./uart_rw -h

```
root@IMX8MP-Tronlong:~# ./uart_rw -h
Usage: ./uart_rw [options]

Options:
 -d | --device        Device such as '/dev/ttyS0'
 -r | --read          Read
 -w | --write         Write
 -l | --loopback      loopback test
 -s | --size          Read size
 -v | --version       Display version information
 -h | --help          Show help content
 e.g. ./uart_rw -d /dev/ttymxc0 -r -s 256
      ./uart_rw -d /dev/ttymxc1 -w -s 1024
      ./uart_rw -d /dev/ttymxc3 -l -s 1024

root@IMX8MP-Tronlong:~#
```

图 15

执行如下命令，评估板通过 RS232 UART4 串口从上位机读取数据。如需测试 RS485 UART1、RS485 UART3、UART1、UART3 串口，请修改为对应的设备节点即可。

Target# ./uart_rw -d /dev/ttymxc3 -r -s 8

```
root@IMX8MP-Tronlong:~# ./uart_rw -d /dev/ttymxc3 -r -s 8
Mode : read
```

图 16

双击运行“4-软件资料\Tools\Windows\”目录下的串口调试工具 SerialDebug.exe。打开 PC 机设备管理器，确认 RS232 UART4 串口 COM 端口号，并按照下图步骤进行操作或设置，从上位机串口调试工具往评估板发送数据。

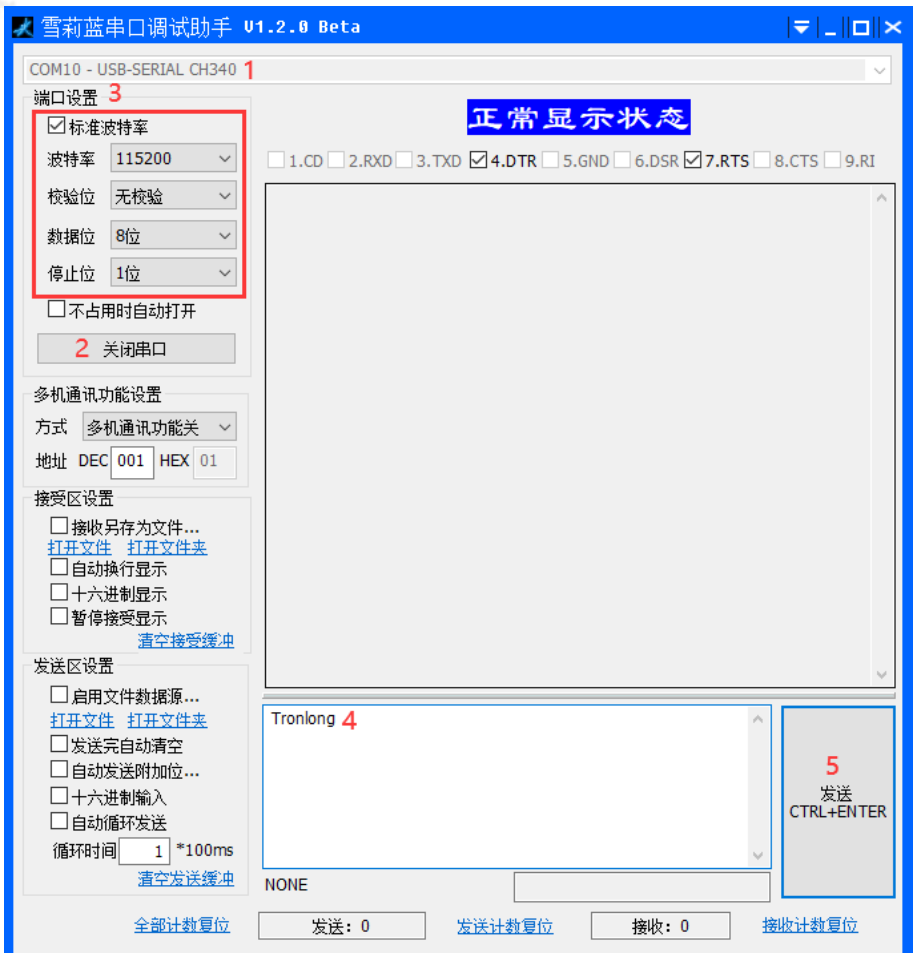

图 17

评估板调试串口终端将会打印从上位机读取到的数据。

```
root@IMX8MP-Tronlong:~# ./uart_rw -d /dev/ttymxc3 -r -s 8
Mode : read
recv: Tronlong
size: 8
root@IMX8MP-Tronlong:~#
```

图 18

执行如下命令，评估板通过 RS232 UART4 串口发送数据至上位机，数据已在程序中定义。

Target# ./uart_rw -d /dev/ttymxc3 -w -s 8

```
root@IMX8MP-Tronlong:~# ./uart_rw -d /dev/ttymxc3 -w -s 8
Mode : write
send size: 8
root@IMX8MP-Tronlong:~#
```

图 19

上位机串口调试工具将会打印从 RS232 UART4 串口接收的数据，如下图所示。

图 20

1.3.3 关键代码

（1） 串口初始化函数。

```c
int init_serial(int *fd, const char *dev) {
    struct termios opt;

    /* open serial device */
    if ((*fd = open(dev, O_RDWR)) < 0) {
        perror("open()");
        return -1;
    }

    /* define termois */
    if (tcgetattr(*fd, &opt) < 0) {
        perror("tcgetattr()");
        return -1;
    }

    opt.c_lflag  &= ~(ICANON | ECHO | ECHOE | ISIG);
    opt.c_oflag  &= ~OPOST;

    /* Character length, make sure to screen out this bit before setting the data bit */
    opt.c_cflag &= ~CSIZE;

    /* No hardware flow control */
    opt.c_cflag &= ~CRTSCTS;

    /* 8-bit data length */
    opt.c_cflag |= CS8;

    /* 1-bit stop bit */
    opt.c_cflag &= ~CSTOPB;

    /* No parity bit */
    opt.c_iflag |= IGNPAR;

    /* Output mode */
    opt.c_oflag = 0;

    /* No active terminal mode */
    opt.c_lflag = 0;

    /* Input baud rate */
    if (cfsetispeed(&opt, B115200) < 0)
        return -1;

    /* Output baud rate */
    if (cfsetospeed(&opt, B115200) < 0)
        return -1;

    /* Overflow data can be received, but not read */
    if (tcflush(*fd, TCIFLUSH) < 0)
        return -1;

    if (tcsetattr(*fd, TCSANOW, &opt) < 0)
        return -1;

    return 0;
}
```

图 21

（2） 串口读写函数。

```c
int serial_write(int *fd, const char *data, size_t size) {
    int ret = write(*fd, data, size);
    if ( ret < 0 ) {
        perror("write");
        tcflush(*fd, TCOFLUSH);
    }

    return ret;
}

int serial_read(int *fd, char *data, size_t size) {
    size_t read_left = size;
    size_t read_size = 0;
    char *read_ptr = data;
    struct timeval timeout = {5, 0};

    memset(data, 0, size);

    fd_set rfds;
    while (!g_quit) {
        FD_ZERO(&rfds);
        FD_SET(*fd, &rfds);
        timeout.tv_sec = 5;
        timeout.tv_usec = 0;
```

图 22

```c
int run_read_mode(char *dev, size_t size) {
    char *buf = NULL;
    int fd = -1;
    int ret = -1;

    ret = init_serial(&fd, dev);
    if (ret < 0) {
        close(fd);
        return -1;
    }

    printf("Mode : read\n");
    if (size <= 0) {
        printf ("Error : Incorrect size settings\n");
        exit(INADEQUATE_CONDITIONS);
    }

    buf = (char*)malloc(size);
    ret = serial_read(&fd, buf, size);
    printf("recv: %s\nsize: %d\n", buf, ret);

    free(buf);
    return 0;
}

int run_write_mode(char *dev, size_t size) {
    int fd = -1;
    int ret = -1;

    ret = init_serial(&fd, dev);
    if (ret < 0) {
        close(fd);
        return -1;
    }
```

图 23

（3） 配置串口读取长度函数。

```c
int run_read_mode(char *dev, size_t size) {
    char *buf = NULL;
    int fd = -1;
    int ret = -1;

    ret = init_serial(&fd, dev);
    if (ret < 0) {
        close(fd);
        return -1;
    }

    printf("Mode : read\n");
    if (size <= 0) {
        printf ("Error : Incorrect size settings\n");
        exit(INADEQUATE_CONDITIONS);
    }

    buf = (char*)malloc(size);
    ret = serial_read(&fd, buf, size);
    printf("recv: %s\nsize: %d\n", buf, ret);

    free(buf);
    return 0;
}
```

图 24

（4） 配置串口读取长度函数。

```c
int run_write_mode(char *dev, size_t size) {
    int fd = -1;
    int ret = -1;

    ret = init_serial(&fd, dev);
    if (ret < 0) {
        close(fd);
        return -1;
    }

    printf("Mode : write\n");
    if (size <= 0) {
        printf("Error : Incorrect size settings\n");
        exit(INADEQUATE_CONDITIONS);
    }

    int i = 0;
    char context;
    size_t write_size = 0;
    while (!g_quit) {
        if (i > 7)
            i = 0;
        context = (char)('0' + i);

        write_size += serial_write(&fd, &context, sizeof(context));
        i ++;

        if (size == write_size)
            break;
    }

    printf("send size: %ld\n", write_size);
    return 0;
}
```

图 25

（5） 回环测试。

```c
int run_loopback_test(char *dev, size_t size) {
    int fd;
    int ret;
    size_t buf_size;
    int serial_buf_size;

    ret = init_serial(&fd, dev);
    if (ret < 0)
        return -1;

    printf("Start uart loopback testing.\n");

    /* Serial port buffer size generally defaults to 2k - 4k */
    char *write_buf = (char*)malloc(size);
    char *read_buf = (char*)malloc(size);

    buf_size = size;
    while (buf_size > 0)
    {
        if(buf_size > 1024) {
            serial_buf_size = 1024;
        } else {
            serial_buf_size = buf_size;
        }
```

图 26

1.4 lcd_demo 案例

1.4.1 案例功能

本案例基于 Framebuffer 向显示设备写入对应数据，实现显示输出横向颜色带（黑、红、紫、黄、绿、青、蓝、白），同时改变背光亮度。

程序流程如下图所示：

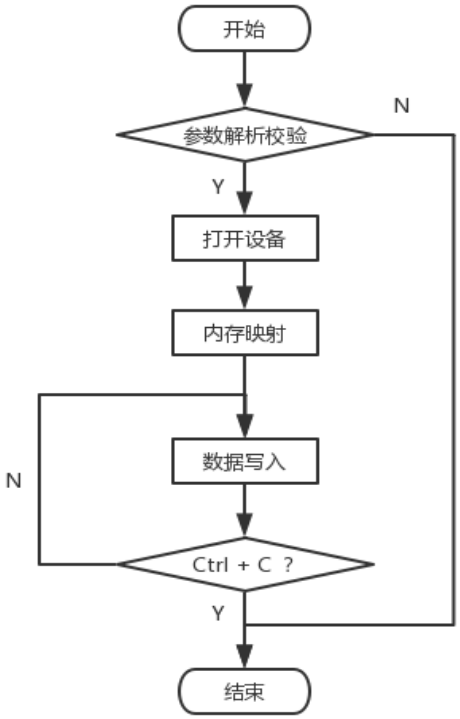

图 27

LCD 设备节点为"/dev/fb0"，背光控制设备节点为"/sys/class/backlight/backlight"，通过向背光控制设备节点写入不同亮度值来控制其亮度。

1.4.2 操作说明

案例支持 LVDS LCD、MIPI LCD 和 HDMI 显示屏显示，本次操作以 LVDS LCD 为例进行演示。

（1） 请将 10.4 英寸 LVDS 显示屏（群创 G104XCE-L01）与评估板的 LVDS LCD（显示）、RES TS（触摸）、BACK LIGHT（背光）接口正确连接，硬件连接如下所示。

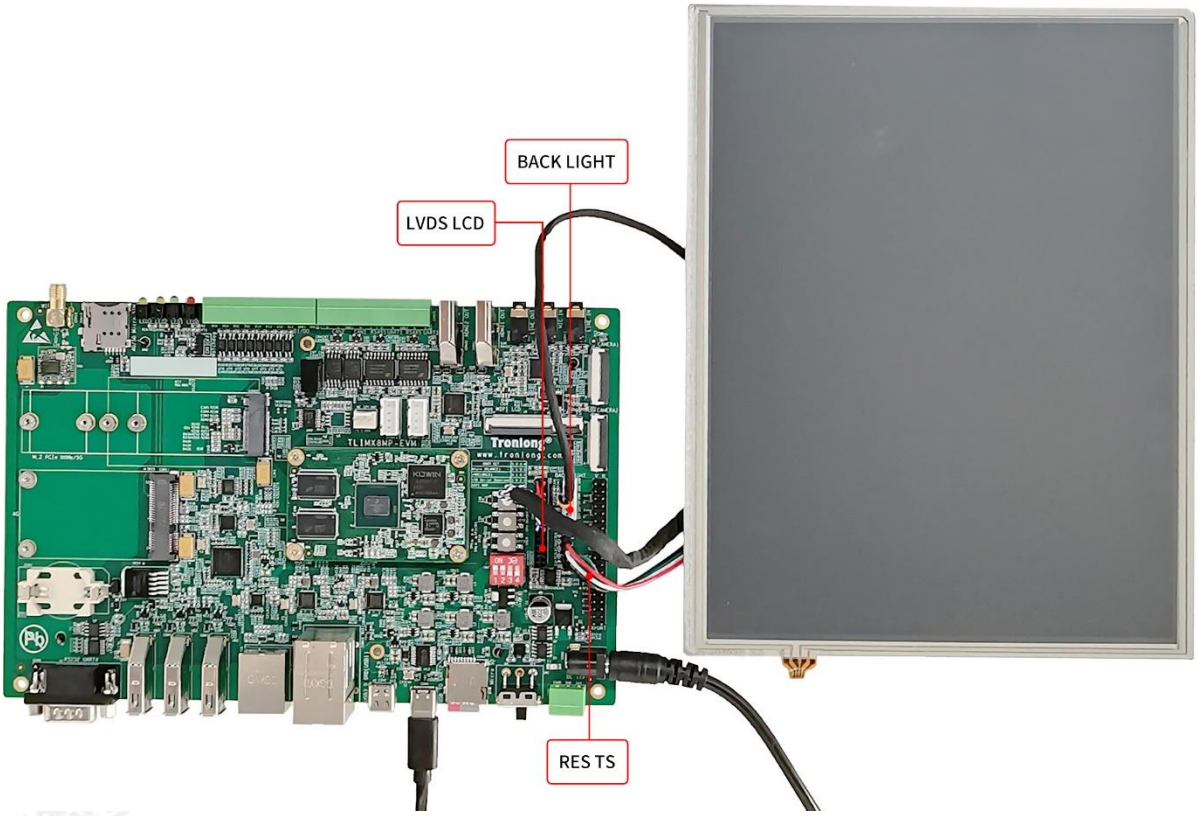

图 28

（2） 由于 Linux 系统默认支持 HDMI1 OUT、HDMI2 OUT 接口显示，因此请执行如下命令切换为支持 LVDS 显示功能的设备树文件。

Target# cd /run/media/mmcblk1p1/

Target# cp tlimx8mp-evm.dtb tlimx8mp-evm-bak.dtb

Target# cp tlimx8mp-evm-lvds.dtb tlimx8mp-evm.dtb

Target# sync

Target# reboot

```
root@IMX8MP-Tronlong:~# cd /run/media/mmcblk1p1/
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# cp tlimx8mp-evm.dtb tlimx8mp-evm-bak.dtb
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# cp tlimx8mp-evm-lvds.dtb tlimx8mp-evm.dtb
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# sync
root@IMX8MP-Tronlong:/run/media/mmcblk1p1# reboot
```

图 29

（3） 执行以下命令，关闭 Weston 服务，并重启评估板。

Target# systemctl disable weston.service

Target# reboot

```
root@IMX8MP-Tronlong:~# systemctl disable weston.service
Removed /etc/systemd/system/graphical.target.wants/weston.service.
[  255.563606] systemd-sysv-generator[955]: SysV service '/etc/init.d/netopeer2-server' lack
s a native systemd unit file. Automatically generating a unit file for compatibility. Please
 update package to include a native systemd unit file, in order to make it more safe and rob
ust.
[  255.563735] systemd-sysv-generator[955]: SysV service '/etc/init.d/save-rtc.sh' lacks a n
ative systemd unit file. Automatically generating a unit file for compatibility. Please upda
te package to include a native systemd unit file, in order to make it more safe and robust.
[  255.564348] systemd-sysv-generator[955]: SysV service '/etc/init.d/sendsigs' lacks a nati
ve systemd unit file. Automatically generating a unit file for compatibility. Please update
package to include a native systemd unit file, in order to make it more safe and robust.
[  255.564644] systemd-sysv-generator[955]: SysV service '/etc/init.d/rc.local' lacks a nati
ve systemd unit file. Automatically generating a unit file for compatibility. Please update
package to include a native systemd unit file, in order to make it more safe and robust.
[  255.565242] systemd-sysv-generator[955]: SysV service '/etc/init.d/sysrepo-plugind' lacks
 a native systemd unit file. Automatically generating a unit file for compatibility. Please
update package to include a native systemd unit file, in order to make it more safe and robu
st.
[  255.589977] systemd-sysv-generator[955]: SysV service '/etc/init.d/umountfs' lacks a nati
ve systemd unit file. Automatically generating a unit file for compatibility. Please update
package to include a native systemd unit file, in order to make it more safe and robust.
[  255.590379] systemd-sysv-generator[955]: SysV service '/etc/init.d/single' lacks a native
 systemd unit file. Automatically generating a unit file for compatibility. Please update pa
ckage to include a native systemd unit file, in order to make it more safe and robust.
[  255.590501] systemd-sysv-generator[955]: SysV service '/etc/init.d/sysrepod' lacks a nati
ve systemd unit file. Automatically generating a unit file for compatibility. Please update
package to include a native systemd unit file, in order to make it more safe and robust.
[  255.590913] systemd-sysv-generator[955]: SysV service '/etc/init.d/reboot' lacks a native
 systemd unit file. Automatically generating a unit file for compatibility. Please update pa
ckage to include a native systemd unit file, in order to make it more safe and robust.
[  255.591014] systemd-sysv-generator[955]: SysV service '/etc/init.d/halt' lacks a native s
ystemd unit file. Automatically generating a unit file for compatibility. Please update pack
age to include a native systemd unit file, in order to make it more safe and robust.
[  255.591256] systemd-sysv-generator[955]: SysV service '/etc/init.d/sysrepo-init' lacks a
native systemd unit file. Automatically generating a unit file for compatibility. Please upd
ate package to include a native systemd unit file, in order to make it more safe and robust.
[  255.591365] systemd-sysv-generator[955]: SysV service '/etc/init.d/sysrepo-tsnd' lacks a
native systemd unit file. Automatically generating a unit file for compatibility. Please upd
ate package to include a native systemd unit file, in order to make it more safe and robust.
[  255.591726] systemd-sysv-generator[955]: SysV service '/etc/init.d/umountnfs.sh' lacks a
native systemd unit file. Automatically generating a unit file for compatibility. Please upd
ate package to include a native systemd unit file, in order to make it more safe and robust.
root@IMX8MP-Tronlong:~# reboot
```

图 30

（4） 将本案例 bin 目录下的可执行程序 lcd_demo 拷贝至评估板文件系统，进入可执行程序所在目录，执行如下命令，查看程序参数信息。

Target# ./lcd_demo -h

```
root@IMX8MP-Tronlong:~# ls lcd_demo
lcd_demo
root@IMX8MP-Tronlong:~# ./lcd_demo -h
Usage: lcd_demo [options]

Options:
 -d | --device        Specify the backlight node (Default: /sys/class/backlight/backlight)
 -v | --version       Display version information
 -h | --help          Show help content

 e.g. ./lcd_demo -d /sys/class/backlight/pwm-backlight

root@IMX8MP-Tronlong:~#
```

图 31

（5） 执行如下命令运行程序，即可看到 LVDS 显示屏显示横向颜色带（黑、红、紫、黄、绿、青、蓝、白），且循环翻转显示，间隔为 3s；同时 LVDS 屏背光亮度从 100%、

80%、60%、40%、20%、0%循环改变，每级亮度保持时间为 0.5s。

Target# ./lcd_demo -d /sys/class/backlight/lvds_backlight

```
root@IMX8MP-Tronlong:~# ./lcd_demo -d /sys/class/backlight/lvds_backlight
Actual backlight is:100
Max backlight is:100
1024x768, 32bpp
```

图 32

图 33

图 34

可按下"Ctrl + C"终止程序。

1.4.3 关键代码

（1） 背光亮度控制函数。

```
/* Check the backlight device node exists */
bool check_backlight_dev(char *backlight_dev) {
    if(access(backlight_dev, F_OK) == 0) {
        return true;
    } else {
        return false;
    }
}

int get_actual_backlight(char *backlight_dev) {
    FILE *fp;
    char cmd[CMD_SIZE];
    char buf[BUF_SIZE];

    snprintf(cmd, CMD_SIZE, "cat %s/actual_brightness", backlight_dev);

    fp = popen(cmd, "r");

    fgets(buf, sizeof(buf), fp);

    pclose (fp);
    return atoi(buf);
}
```

图 35

```c
int get_max_backlight(char *backlight_dev) {
    FILE *fp;
    char cmd[CMD_SIZE];
    char buf[BUF_SIZE];

    snprintf(cmd, CMD_SIZE, "cat %s/max_brightness", backlight_dev);

    fp = popen(cmd, "r");

    fgets(buf, sizeof(buf), fp);

    pclose (fp);
    return atoi(buf);
}

int set_brightness(char *backlight_dev, int value) {
    char cmd[CMD_SIZE];

    snprintf(cmd, CMD_SIZE, "echo %d > %s/brightness", value, backlight_dev);
    if ((system(cmd)) != 0)
        return -1;

    return 0;
}

void run_control_backlight(char *backlight_dev, int max_value) {
    /* loop to brighten backlight */
    set_brightness(backlight_dev, max_value);
    usleep(500000);

    set_brightness(backlight_dev, max_value * 0.8);
    usleep(500000);

    set_brightness(backlight_dev, max_value * 0.6);
    usleep(500000);

    set_brightness(backlight_dev, max_value * 0.4);
    usleep(500000);

    set_brightness(backlight_dev, max_value * 0.2);
    usleep(500000);

    set_brightness(backlight_dev, 0);
    usleep(500000);
}
```

图 36

（2） 颜色带显示函数。

```c
/* Draw RGB pixel to framebuffer. */
void drawPixel(char *fbp, unsigned long location, PIXEL *pixel, int bpp) {
    unsigned short rgb = 0;
    unsigned int r = 0, g = 0, b = 0;
    switch (bpp) {
    case 32:
        *(fbp + location) = pixel->blue;
        *(fbp + location + 1) = pixel->green;
        *(fbp + location + 2) = pixel->red;
        *(fbp + location + 3) = pixel->trans;
        break;

    case 24:
        *(fbp + location) = pixel->blue;
        *(fbp + location + 1) = pixel->green;
        *(fbp + location + 2) = pixel->red;
        break;

    case 16:
        r = (pixel->red * 0x1f) / 0xff;
        g = (pixel->green * 0x3f) / 0xff;
        b = (pixel->blue * 0x1f) / 0xff;
        rgb = (r << 11) | (g << 5) | b;
        *((unsigned short*)(fbp + location)) = rgb;
        break;

    default:
        printf("Error: No fit bpp\n");
        break;
    }
}
```

图 37

```c
void run_draw(char *fbp, struct fb_var_screeninfo *vinfo, struct fb_fix_screeninfo *finfo) {
    /* Display test */
    PIXEL pixel = { 0x00, 0x00, 0x00, 0x00 };

    int section = vinfo->yres / 8;
    int factor = 0;
    static bool reversal = false;
    unsigned long location = 0;
    int x = 0, y = 0;

    reversal = !reversal;
    for ( y = 0; y < vinfo->yres; y++ ) {
        factor = y / section;
        if ( reversal ) {
            /* reverse the color bar. */
            factor = 7 - factor;
        }
        for ( x = 0; x < vinfo->xres; x++ ) {
            location = (x+vinfo->xoffset) * (vinfo->bits_per_pixel / 8) + (y+vinfo->yoffset) * finfo->line_length;
            /* Red border */
            if (x == 0 || x >= vinfo->xres - 1 || y == 0 || y >= vinfo->yres - 1) {
                pixel.red = 0xff;
                pixel.green = 0x00;
                pixel.blue = 0x00;
                pixel.trans = 0x00;
            }
            /* Black border inside red border. */
            else if (x <= 3 || x >= vinfo->xres - 4 || y <= 3 || y >= vinfo->yres - 4) {
                pixel.red = 0x00;
                pixel.green = 0x00;
                pixel.blue = 0x00;
                pixel.trans = 0x00;
            }
            else {
                switch (factor) {
                case 0: // Black
                    pixel.red = 0x00;
                    pixel.green = 0x00;
                    pixel.blue = 0x00;
                    pixel.trans = 0x00;
```

图 38

（3） 相关逻辑。

```
    /* Get fixed screen information */
    if (ioctl(fbfd, FBIOGET_FSCREENINFO, &finfo)) {
        printf("Error reading fixed information.\n");
        close(fbfd);
        exit(INADEQUATE_CONDITIONS);
    }

    /* Get variable screen information */
    if (ioctl(fbfd, FBIOGET_VSCREENINFO, &vinfo)) {
        printf("Error reading variable information.\n");
        close(fbfd);
        exit(INADEQUATE_CONDITIONS);
    }
    else
        printf("%dx%d, %dbpp\n", vinfo.xres, vinfo.yres, vinfo.bits_per_pixel);

    /* Map the device to memory */
    fbp = (char *)mmap(0, finfo.smem_len, PROT_READ | PROT_WRITE, MAP_SHARED, fbfd, 0);
    if (fbp == MAP_FAILED) {
        printf("Error: failed to map framebuffer device to memory.\n");
        close(fbfd);
        exit(INADEQUATE_CONDITIONS);
    }

    /*
    1. Show color bar of black,red,purple,yellow,green,light blue,blue,white, from top to bottom.
    2. Control backlight brightness
    */
    while (!g_quit) {
        run_draw(fbp, &vinfo, &finfo);
        run_control_backlight(backlight_dev, max_value);
    }

    /* get back to actual backlight */
    set_brightness(backlight_dev, actual_value);

    /* Release Framebuffer */
    munmap(fbp, finfo.smem_len);
    close(fbfd);
    return 0;
}
```

图 39

1.5 tcp_udp 案例

1.5.1 案例功能

本案例主要实现客户端(client)与服务端(server)的文本数据相互收发功能。本案例包含 4 个程序：

（1） tcp_server：TCP 服务端测试程序。

（2） tcp_client：TCP 客户端测试程序。

（3） udp_server：UDP 服务端测试程序。

（4） udp_client：UDP 客户端测试程序。

程序流程如下图所示：

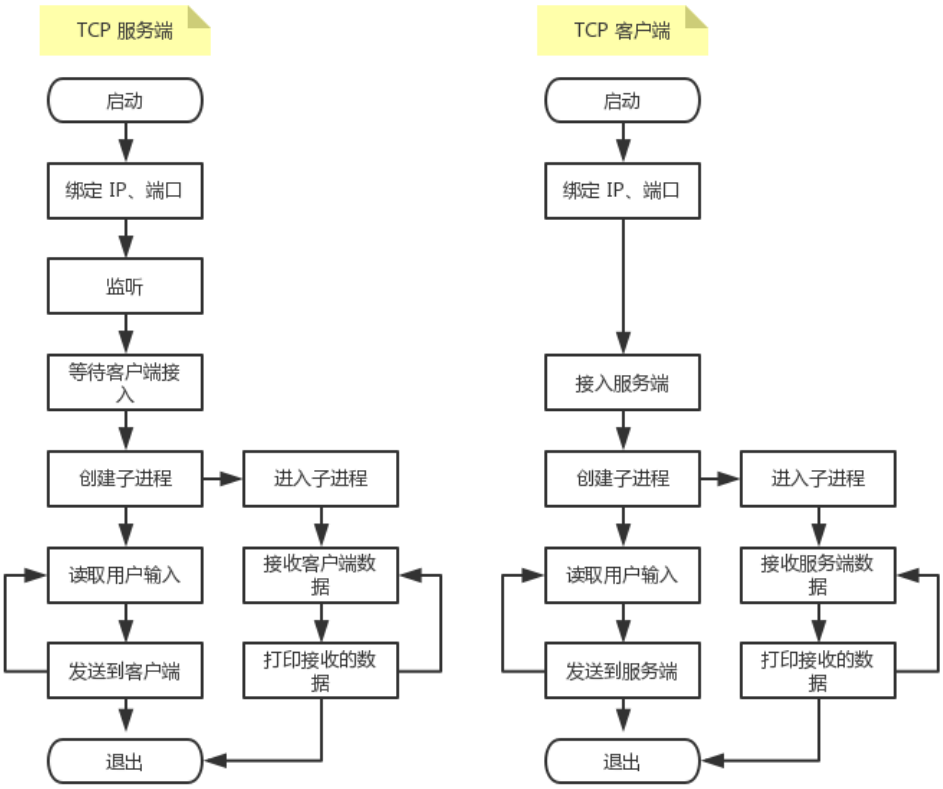
TCP 服务端
启动
绑定 IP、端口
监听
等待客户端接入
创建子进程
进入子进程
读取用户输入
接收客户端数据
发送到客户端
打印接收的数据
退出
TCP 客户端
启动
绑定 IP、端口
接入服务端
创建子进程
进入子进程
读取用户输入
接收服务端数据
发送到服务端
打印接收的数据
退出

图 40 TCP 通信

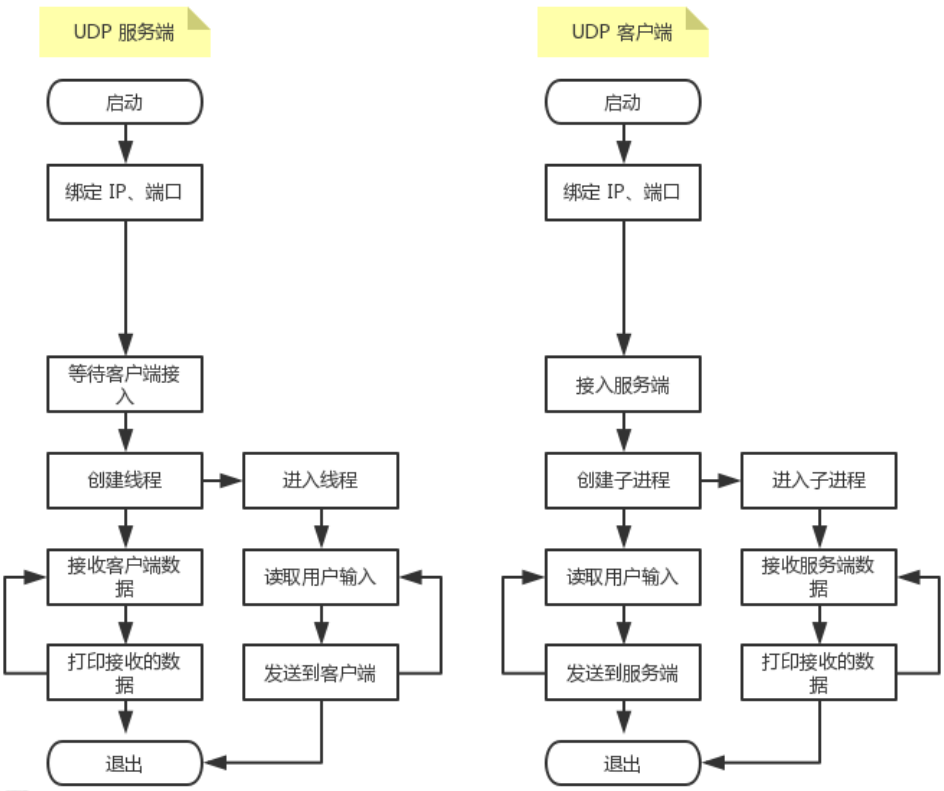

图 41 UDP 通信

服务端和客户端程序均可在评估板、PC 机 Ubuntu 系统上运行。当服务端与客户端程序均在评估板上运行时，可通过 127.0.0.1 进行本地回环测试，无需经过路由器。为方便测试，本章节采用评估板本地回环测试，bin 目录下的 4 个文件均为 ARM 端可执行程序。

1.5.2 操作说明

请将本案例 bin 目录下的 4 个可执行程序拷贝至评估板文件系统任意目录下。

在 Ubuntu 中执行如下命令使用 OpenSSH 登录评估板文件系统，如下图所示。

Host# sudo ssh root@192.168.13.49 //192.168.13.49 为评估板 IP，请根据实际情况修改

```
tronlong@tronlong-virtual-machine:~$ sudo ssh root@192.168.13.49
[sudo] password for tronlong:
The authenticity of host '192.168.13.49 (192.168.13.49)' can't be established.
RSA key fingerprint is SHA256:jbU+hs/iOfDUZ6e8iX8TKw2Ej32GSrKAYUWgBJKHWuk.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.13.49' (RSA) to the list of known hosts.
root@IMX8MP-Tronlong:~# cd tcp_udp/bin/
root@IMX8MP-Tronlong:~/tcp_udp/bin# ls
tcp_client  tcp_server  udp_client  udp_server
root@IMX8MP-Tronlong:~/tcp_udp/bin#
```

图 42

（1） TCP 通信测试

在可执行程序所在目录执行如下命令运行 TCP 服务端和客户端程序。2233 为服务端程序指定的端口号，客户端程序端口号参数需与服务端程序一致。

Target# ./tcp_server 2233 //TCP 服务端命令

Host# ./tcp_client 127.0.0.1 2233 //TCP 客户端命令

程序执行后，客户端将会连接服务端。在服务端（评估板）输入字符串"Tronlong"，按下回车键即可在客户端显示对应内容。在客户端(Ubuntu)输入字符串"Hello Tronlong"，按下回车键即可在服务端显示对应内容，如下图所示。

```
root@IMX8MP-Tronlong:~/tcp_udp/bin# ls
tcp_client  tcp_server  udp_client  udp_server
root@IMX8MP-Tronlong:~/tcp_udp/bin# ./tcp_server 2233
Waiting for client connectionn...
Client accepted : 127.0.0.1:57890
Tronlong
[Recv from client 0.0.0.0:2233] : Hello,Tronlong
```

图 43 TCP 服务端

```
root@IMX8MP-Tronlong:~/tcp_udp/bin# ./tcp_client 127.0.0.1 2233
connect server(IP:127.0.0.1 Port: 2233).
[Recv from server 127.0.0.1:2233] : Tronlong
Hello,Tronlong
```

图 44 TCP 客户端

（2） UDP 通信测试

在可执行程序所在目录先后执行如下命令运行 UDP 服务端和客户端程序。2233 为服务端程序指定的端口号，客户端程序端口号参数需与服务端程序一致。

Host# ./udp_server 2233 //UDP 服务端命令

Target# ./udp_client 127.0.0.1 2233 //UDP 客户端命令

程序执行后，客户端将不会自动连接服务端。服务端在收到客户端信息前无法得知客户端的存在，因此需客户端先向服务端发送信息。

在客户端（评估板）输入字符串"Tronlong"，按下回车键即可在服务端显示对应内容。在服务端(Ubuntu)输入字符串"Hello Tronlong"，按下回车键即可在客户端显示对应内容，如下图所示。

```
root@IMX8MP-Tronlong:~/tcp_udp/bin# ./udp_server 2233
Waiting for client connectionn
[Recv from client 127.0.0.1:42263] : Tronlong
Hello,Tronlong
```

图 45 UDP 服务端

```
root@IMX8MP-Tronlong:~/tcp_udp/bin# ./udp_client 127.0.0.1 2233
Tronlong
[Recv from server 127.0.0.1:2233] : Hello,Tronlong
```

图 46 UDP 客户端

如需在 PC 端 Ubuntu 系统运行服务端或客户端程序，请将案例 src 源码目录拷贝至 Ubuntu 工作目录。进入源码目录后配置交叉编译工具链环境变量，然后执行 make 命令，即可在当前目录下生成 x86 端可执行程序。在不同终端运行服务端或客户端程序的命令类似，但 127.0.0.1 需使用服务器 IP 地址替代。

Host# source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

Host# make

```
tronlong@tronlong-virtual-machine:~/base-demos/tcp_udp/src$ source /home/tronlong/SDK/environ
ment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/base-demos/tcp_udp/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SO
URCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroo
ts/armv8a-poky-linux -c tcp_client.c -o tcp_client.o  -Wall -g -O2
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SO
URCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroo
ts/armv8a-poky-linux -o tcp_client tcp_client.o  -lpthread
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SO
URCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroo
ts/armv8a-poky-linux -c tcp_server.c -o tcp_server.o  -Wall -g -O2
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SO
URCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroo
ts/armv8a-poky-linux -o tcp_server tcp_server.o  -lpthread
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SO
URCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroo
ts/armv8a-poky-linux -c udp_client.c -o udp_client.o  -Wall -g -O2
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SO
URCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroo
ts/armv8a-poky-linux -o udp_client udp_client.o  -lpthread
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SO
URCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroo
ts/armv8a-poky-linux -c udp_server.c -o udp_server.o  -Wall -g -O2
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SO
URCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroo
ts/armv8a-poky-linux -o udp_server udp_server.o  -lpthread
tronlong@tronlong-virtual-machine:~/base-demos/tcp_udp/src$ ls
makefile      tcp_client.o  tcp_server.o  udp_client.o  udp_server.o
tcp_client    tcp_server    udp_client    udp_server
tcp_client.c  tcp_server.c  udp_client.c  udp_server.c
tronlong@tronlong-virtual-machine:~/base-demos/tcp_udp/src$
```

图 47

1.5.3 关键代码

以 TCP 通信程序为例。

（1） tcp_client.c

注意源码中的 struct sockaddr_in、socket、connect、fgets、send、recv 等数据结构和系统调用的使用。

（2） tcp_server.c

注意源码中的 struct sockaddr_in、socket、connect、fgets、send、recv、bind、listen、accept 等数据结构和系统调用的使用。

1.6 can_echo 案例

1.6.1 案例功能

本案例主要实现通过绑定一个 CAN 总线接口用于接收数据，当 CAN 端口接收到数据后，将数据重新发送到 CAN 总线接口。

程序流程如下图所示：

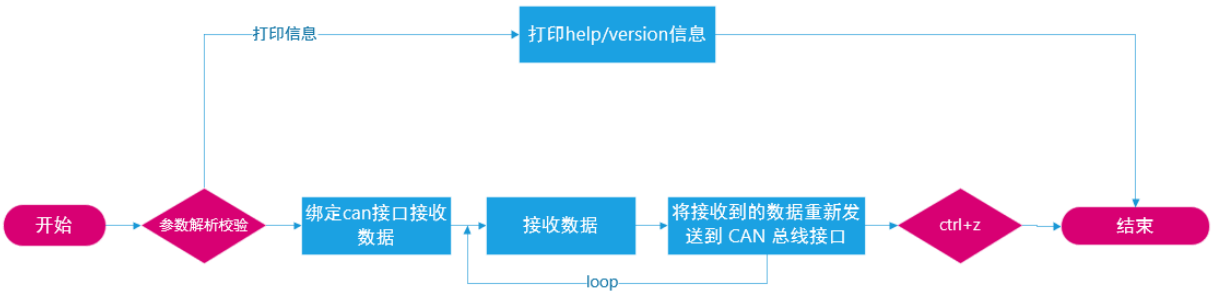

图 48

1.6.2 操作说明

使用 USB-CAN 分析仪（型号：广成科技 USBCAN-II FD）连接评估板 CAN1 接口（或 CAN2 接口）和 PC 机 USB 接口，本次测试以 CAN1 接口进行演示，硬件连接如下图所示。

备注：CAN1 接口设备节点为：can0，CAN2 接口设备节点为：can1。

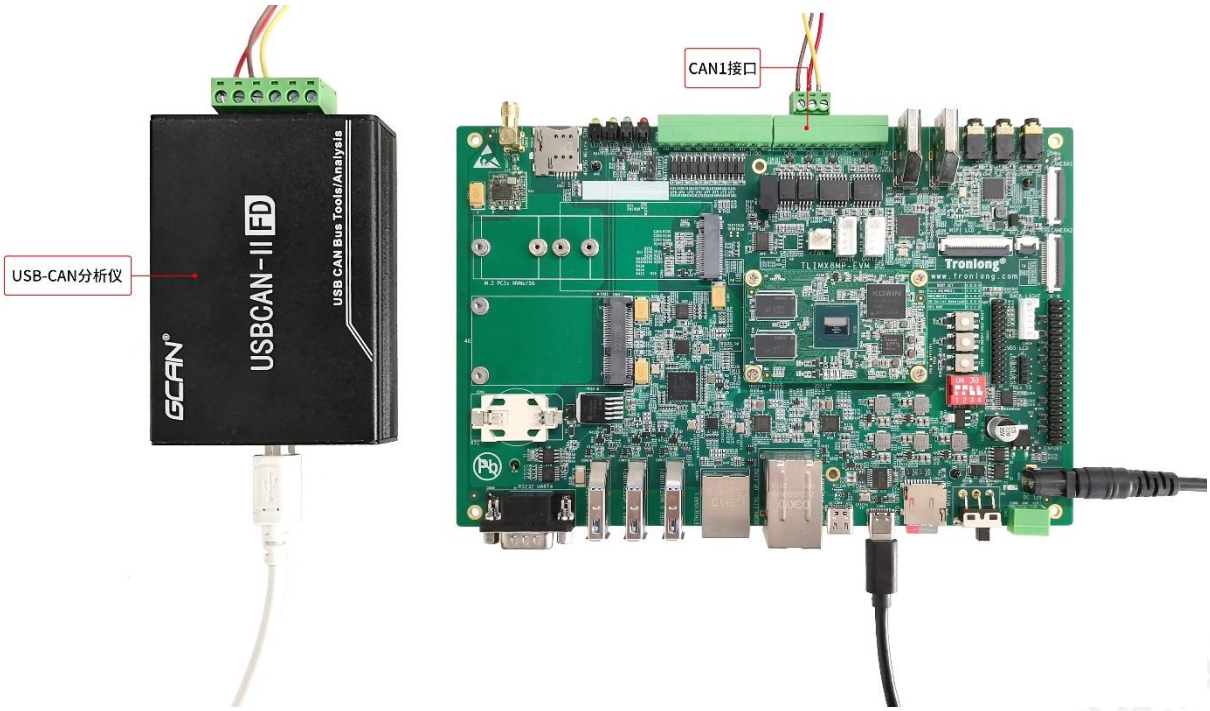

图 49

评估板与 USB-CAN 分析仪的硬件连接关系如下表所示。

表 2

评估板	USB-CAN 分析仪
CAN1 接口 H 端子	H
CAN1 接口 L 端子	L
GNDI 端子	G

参照调试工具安装文档安装 GCAN Tools 调试软件。在 PC 端双击打开 GCAN Tools 软件，选择设备类型，然后点击“打开设备”。根据 USB-CAN 分析仪实际硬件连接，选择 CAN1 或 CAN2。等待软件检测到设备后，设置波特率为"1000K"，然后点击“确定”，如下图所示。

图 50

图 51

进入 GCAN Tools 软件发送界面，取消"CAN FD"选项，USB-CAN 分析仪连接成功后将会显示 Connected 状态，如下图所示。

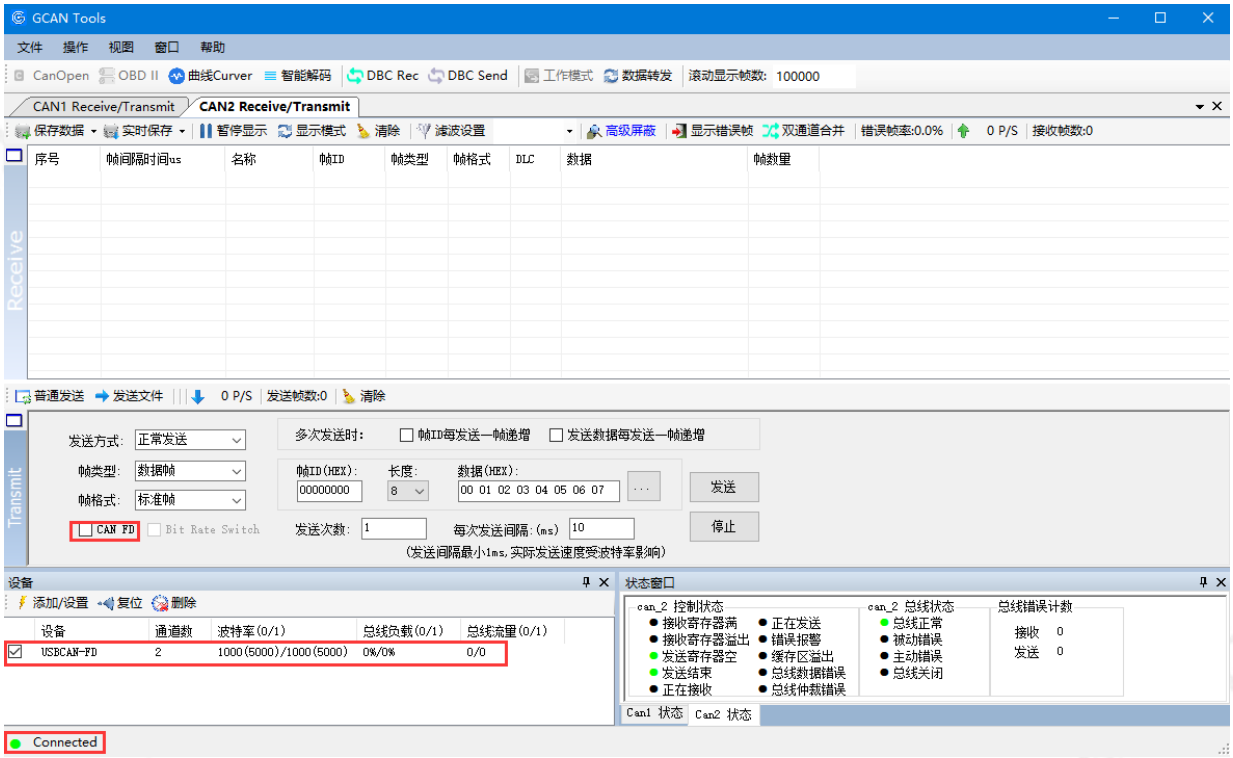

图 52

进入评估板文件系统，执行如下命令配置 CAN 总线比特率为 1Mbps，并启动 CAN 总线。

Target# ip link set can0 down

Target# ip link set can0 type can bitrate 1000000 dbitrate 1000000 fd on

Target# ip link set can0 up

```
root@IMX8MP-Tronlong:~# ip link set can0 down
root@IMX8MP-Tronlong:~# ip link set can0 type can bitrate 1000000 dbitrate 1000000 fd on
root@IMX8MP-Tronlong:~# ip link set can0 up
[   91.014169] flexcan 308c0000.can can0: Data brp=2 and brp=1 don't match, this may result in a
phase error. Consider using different bitrate and/or data bitrate.
[   91.028833] IPv6: ADDRCONF(NETDEV_CHANGE): can0: link becomes ready
root@IMX8MP-Tronlong:~#
```

图 53

将本案例 bin 目录下的可执行程序 can_echo 拷贝至评估板文件系统，进入可执行程序所在目录，执行如下命令查看程序参数信息。

Target# ./can_echo -help

```
root@IMX8MP-Tronlong:~# ./can_echo -help
Usage: can_echo <can-interface> [<can-interface-out>] [Options]

Send all messages received on <can-interface> to <can-interface-out>
If <can-interface-out> is omitted, then <can_interface> is used for sending

Options:
 -f, --family=FAMILY   Protocol family (default PF_CAN = 29)
 -t, --type=TYPE       Socket type, see man 2 socket (default SOCK_RAW = 3)
 -p, --protocol=PROTO  CAN protocol (default CAN_RAW = 1)
 -v, --verbose         be verbose
 -h, --help            this help
     --version         print version information and exit
root@IMX8MP-Tronlong:~#
```

图 54

执行如下命令绑定 CAN1 接口，并接收由 GCAN Tools 发出的数据，然后将接收到的数据重新发送出去。在 GCAN Tools 中输入数据并点击发送按钮，可看到有两帧数据，一帧是发送数据，另一帧是接收数据。

Target# ./can_echo -v can0

```
root@IMX8MP-Tronlong:~# ./can_echo -v can0
interface-in = can0, interface-out = can0, family = 29, type = 3, proto = 1
```

图 55

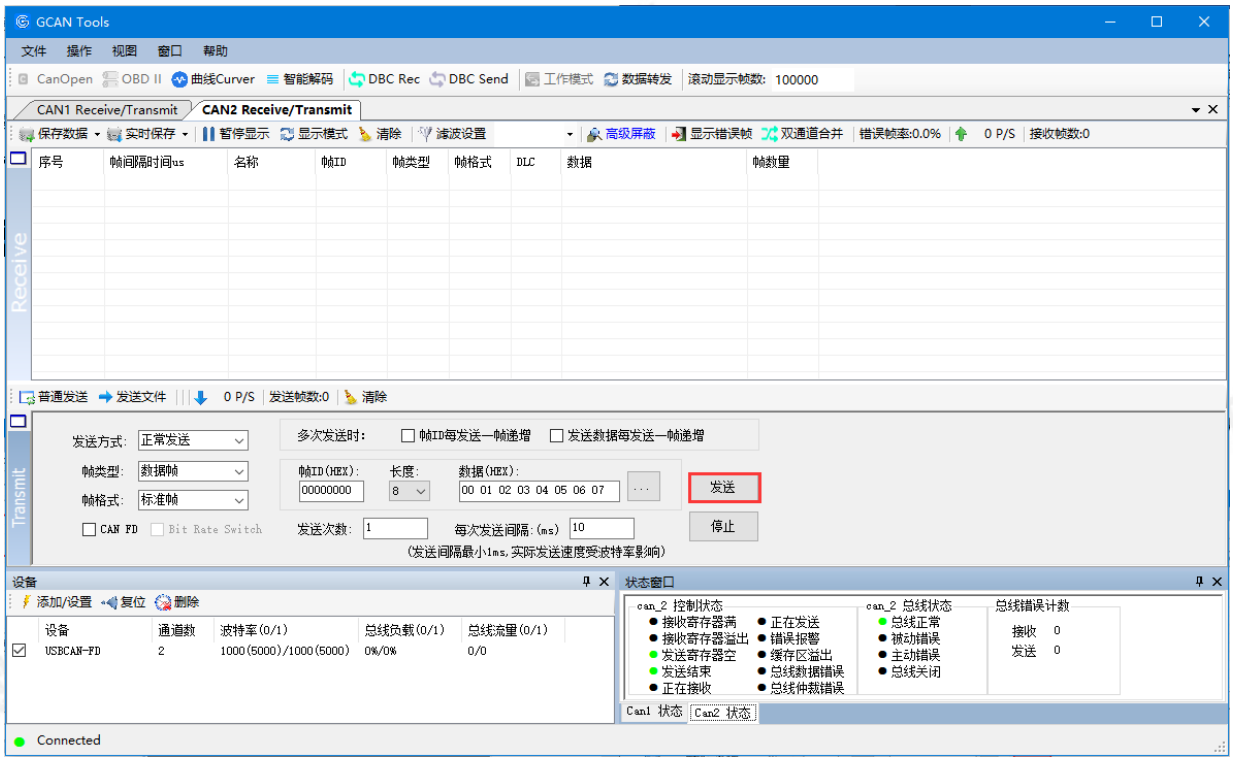

图 56

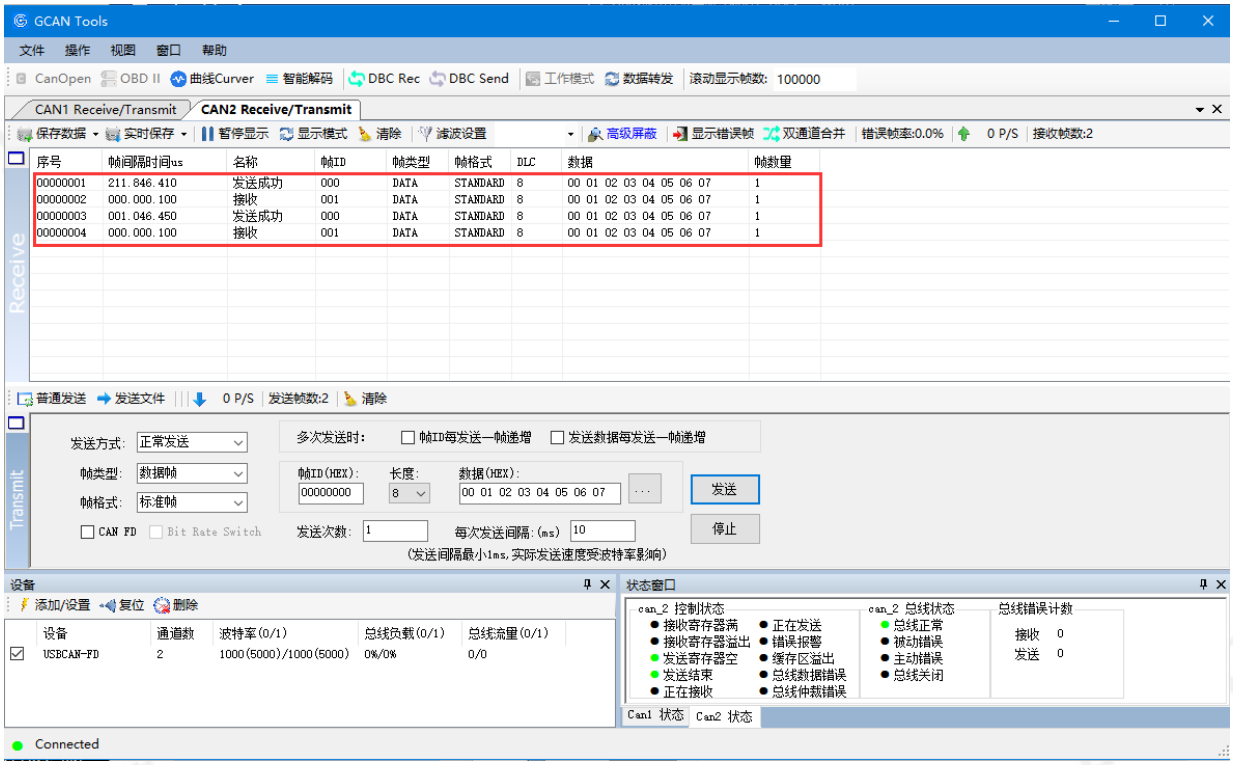

图 57

```
root@IMX8MP-Tronlong:~# ./can_echo -v can0
interface-in = can0, interface-out = can0, family = 29, type = 3, proto = 1
0000: [8] 00 01 02 03 04 05 06 07
0000: [8] 00 01 02 03 04 05 06 07
```

图 58

可按"Ctrl + Z"暂停程序，并执行如下命令退出程序。

Target# killall -9 can_echo

```
root@IMX8MP-Tronlong:~# ./can_echo -v can0
interface-in = can0, interface-out = can0, family = 29, type = 3, proto = 1
0000: [8] 00 01 02 03 04 05 06 07
0000: [8] 00 01 02 03 04 05 06 07
^Z
[4]+  Stopped(SIGTSTP)        ./can_echo -v can0
root@IMX8MP-Tronlong:~#
```

图 59

```
root@IMX8MP-Tronlong:~# killall -9 can_echo
[1]+  Killed                  ./can_echo -v can0
root@IMX8MP-Tronlong:~#
```

图 60

1.6.3 关键代码

（1） 使用 socket 监听 CAN 接口。

```
    for (i = 0; i <= out; i++) {
        if ((s[i] = socket(family, type, proto)) < 0) {
            perror("socket");
            return 1;
        }

        addr[i].can_family = family;
        strcpy(ifr[i].ifr_name, intf_name[i]);
        ioctl(s[i], SIOCGIFINDEX, &ifr[i]);
        addr[i].can_ifindex = ifr[i].ifr_ifindex;

        if (bind(s[i], (struct sockaddr *)&addr[i], sizeof(addr)) < 0) {
            perror("bind");
            return 1;
        }
        setsockopt(s[i], SOL_CAN_RAW, CAN_RAW_FD_FRAMES,
                &can_fd, sizeof(can_fd));
    }
```

图 61

（2） 将从 CAN 接口接收到的数据重新发送出去。

```
    while (running) {
        if ((nbytes = read(s[0], &frame, sizeof(frame))) < 0) {
            perror("read");
            return 1;
        }
        if (verbose) {
            printf("%04x: ", frame.can_id);
            if (frame.can_id & CAN_RTR_FLAG) {
                printf("remote request");
            } else {
                printf("[%d]", frame.can_dlc);
                for (i = 0; i < frame.can_dlc; i++) {
                    printf(" %02x", frame.data[i]);
                }
            }
            printf("\n");
        }
        frame.can_id++;
        ret = write(s[out], &frame, sizeof(frame));
        if(!ret)
            printf("write fail\n");
    }

    return 0;
}
```

图 62

2 Python 开发案例

本章节以两个简单案例演示 Python 的使用方法。系统支持 Python 3.10，相关库位于文件系统"/usr/lib/python3.10/"目录下。

```
root@IMX8MP-Tronlong:~# ls /usr/lib/python3.10/
LICENSE.txt                                  fileinput.py      re.py
__future__.py                                fnmatch.py        reprlib.py
__phello__.foo.py                            fractions.py      rlcompleter.py
__pycache__                                  ftplib.py         runpy.py
_aix_support.py                              functools.py      sched.py
_bootsubprocess.py                           genericpath.py    secrets.py
_collections_abc.py                          getopt.py         selectors.py
_compat_pickle.py                            getpass.py        shelve.py
_compression.py                              gettext.py        shlex.py
_markupbase.py                               glob.py           shutil.py
_osx_support.py                              graphlib.py       signal.py
_py_abc.py                                   gzip.py           site-packages
_pydecimal.py                                hashlib.py        site.py
_pyio.py                                     heapq.py          smtpd.py
_sitebuiltins.py                             hmac.py           smtplib.py
_strptime.py                                 html              sndhdr.py
_sysconfigdata__linux_aarch64-linux-gnu.py   http              socket.py
_threading_local.py                          idlelib           socketserver.py
_weakrefset.py                               imaplib.py        sqlite3
abc.py                                       imghdr.py         sre_compile.py
aifc.py                                      imp.py            sre_constants.py
antigravity.py                               importlib         sre_parse.py
argparse.py                                  inspect.py        ssl.py
ast.py                                       io.py             stat.py
asynchat.py                                  ipaddress.py      statistics.py
asyncio                                      json              string.py
asyncore.py                                  keyword.py        stringprep.py
base64.py                                    lib-dynload       struct.py
bdb.py                                       lib2to3           subprocess.py
binhex.py                                    linecache.py      sunau.py
bisect.py                                    locale.py         symtable.py
bz2.py                                       logging           sysconfig.py
cProfile.py                                  lzma.py           tabnanny.py
calendar.py                                  mailbox.py        tarfile.py
cgi.py                                       mailcap.py        telnetlib.py
cgitb.py                                     mimetypes.py      tempfile.py
chunk.py                                     modulefinder.py   textwrap.py
cmd.py                                       multiprocessing   this.py
code.py                                      netrc.py          threading.py
codecs.py                                    nntplib.py        timeit.py
codeop.py                                    ntpath.py         tkinter
collections                                  nturl2path.py     token.py
colorsys.py                                  numbers.py        tokenize.py
compileall.py                                opcode.py         trace.py
concurrent                                   operator.py       traceback.py
config-3.10-aarch64-linux-gnu                optparse.py       tracemalloc.py
configparser.py                              os.py             tty.py
contextlib.py                                pathlib.py        turtle.py
```

图 63

```
contextvars.py                               pdb.py            turtledemo
copy.py                                      pickle.py         types.py
copyreg.py                                   pickletools.py    typing.py
crypt.py                                     pipes.py          unittest
csv.py                                       pkgutil.py        urllib
ctypes                                       platform.py       uu.py
curses                                       plistlib.py       uuid.py
dataclasses.py                               poplib.py         venv
datetime.py                                  posixpath.py      warnings.py
dbm                                          pprint.py         wave.py
decimal.py                                   profile.py        weakref.py
difflib.py                                   pstats.py         webbrowser.py
dis.py                                       pty.py            wsgiref
distutils                                    py_compile.py     xdrlib.py
doctest.py                                   pyclbr.py         xml
email                                        pydoc.py          xmlrpc
encodings                                    pydoc_data        zipapp.py
ensurepip                                    queue.py          zipfile.py
enum.py                                      quopri.py         zipimport.py
filecmp.py                                   random.py         zoneinfo
root@IMX8MP-Tronlong:~#
```

图 64

2.1 led_flash 案例

2.1.1 案例功能

本案例通过向评估底板用户指示灯 LED 设备节点反复交替写入 1、0 数值，实现 LED 闪烁效果。LED 点亮与熄灭时间均为 0.5s。

程序流程如下图所示：

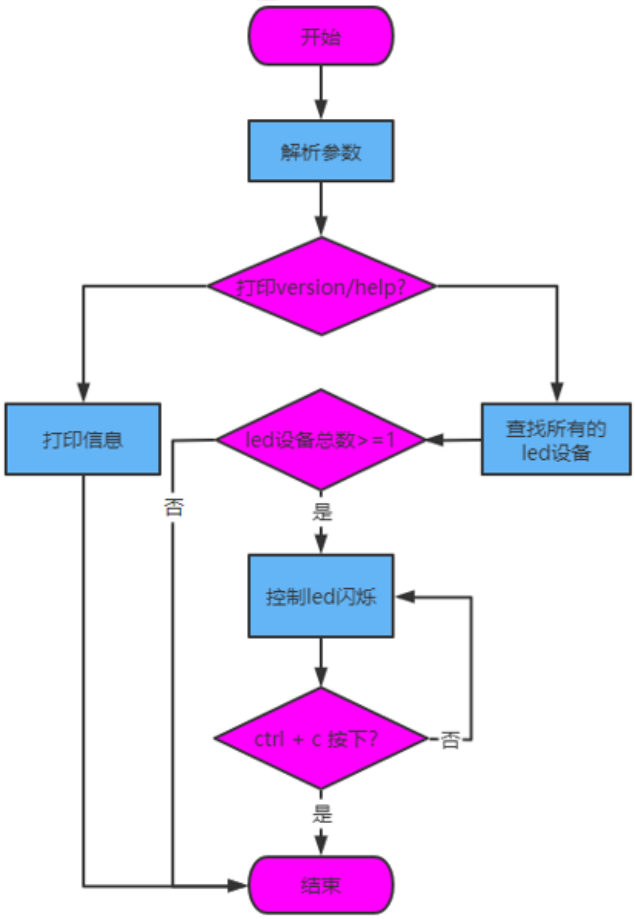

图 65

LED 设备节点为"/sys/class/leds/user-ledX/"目录下的 brightness。

```
root@IMX8MP-Tronlong:~# ls /sys/class/leds/
disk  heartbeat  mmc1::  mmc2::  user-led0
root@IMX8MP-Tronlong:~# ls /sys/class/leds/user-led0
brightness  device  max_brightness  power  subsystem  trigger  uevent
root@IMX8MP-Tronlong:~#
```

图 66

2.1.2 操作说明

将案例目录下的 led_flash.py 脚本文件拷贝至评估板文件系统，并在脚本文件所在目录执行如下命令查看程序参数信息。

Target# ./led_flash.py -h

```
root@IMX8MP-Tronlong:~# ./led_flash.py -h
Usage: ./led_flash.py [options]
Options:
   -v | --version Display version information
   -h | --help    Display help

root@IMX8MP-Tronlong:~#
```

图 67

执行如下命令运行脚本程序，即可看到评估底板上的 LED 进行闪烁。

Target# ./led_flash.py

```
root@IMX8MP-Tronlong:~# ./led_flash.py
find leds:
/sys/class/leds/user-led0

flash led ...
```

图 68

可按下"Ctrl + C"终止程序。

2.1.3 关键代码

（1） 查找所有 LED 设备。

```python
    def enumerate_led(self):
        led_name = "user-led"

        """ find led """
        for filename in os.listdir(self.led_path):
            if led_name in filename:
                self.leds.append(os.path.join(self.led_path, filename))

        if len(self.leds) == 0:
            return False

        """
        led sort, e.g.
        /sys/class/leds/user-led0
        /sys/class/leds/user-led1
        /sys/class/leds/user-led2
        """
        self.leds.sort()

        print ("find leds:")
        for led in self.leds:
            print (led)

        return True
```

图 69

（2） 控制 LED 亮灭。

```python
    """ control led flash """
    def flash_led(self):
        led_num = len(self.leds)

        while not Led.quit_flag:
            """ Turn on leds """
            for i in range(led_num):
                """ Set the led brightness value to 1 to turn on the led """
                ret = os.system("echo 1 > %s/brightness" % (self.leds[i]))

                if ret != 0:
                    print ("Error: Failed to turn on %s") % self.leds[i]

            """ Keep the leds on for 500 ms """
            time.sleep(0.5)

            for i in range(led_num):
                """ Set the led brightness value to 0 to turn off the LED """
                os.system("echo 0 > %s/brightness" % (self.leds[i]))

            """ Keep the leds off for 500 ms """
            time.sleep(0.5)
```

图 70

2.2 key_demo 案例

2.2.1 案例功能

本案例通过监听用户输入按键设备节点状态，检测按键事件。

程序流程如下图所示：

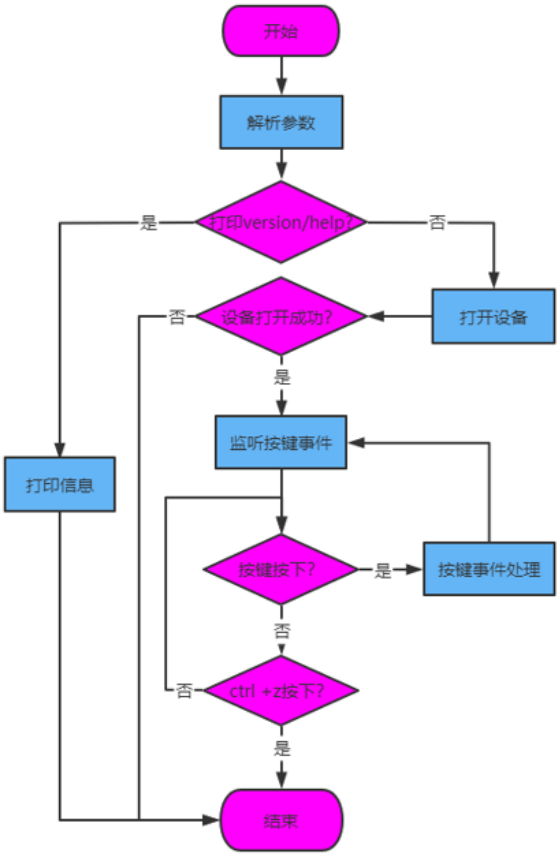

图 71

2.2.2 操作说明

将案例目录下的 key_demo.py 脚本文件拷贝至评估板文件系统，并在脚本文件所在目录执行如下命令查看程序参数信息。

Target# ./key_demo.py -h

```
root@IMX8MP-Tronlong:~# ./key_demo.py -h
Usage: ./key_demo.py [options]
Options:
   -d | --device  Device name (such as: /dev/input/event0)
   -v | --version Display version information
   -h | --help    Display help

e.g. :
       ./key_demo.py -d /dev/input/event0

root@IMX8MP-Tronlong:~#
```

图 72

执行如下命令运行脚本程序，串口终端将打印提示信息。再按下评估板用户输入按键，

程序将检测到按键事件，并打印按键状态信息。

Target# ./key_demo.py -d /dev/input/event0

```
root@IMX8MP-Tronlong:~#  ./key_demo.py -d /dev/input/event0
Please press the key to test.

User key0 pressed!

User key0 pressed!

User key0 pressed!
```

图 73

可按下"Ctrl + Z"暂停程序，输入以下命令终止程序。

Target# killall -9 key_demo.py

```
root@IMX8MP-Tronlong:~# killall -9 key_demo.py
root@IMX8MP-Tronlong:~#
[1]+  Killed                  ./key_demo.py -d /dev/input/event0
root@IMX8MP-Tronlong:~#
```

图 74

2.2.3 关键代码

（1） 打开按键设备。

```
def open_device(self, key_dev):
    try:
        """ open key device, if success, return True"""
        self.key_dev = evdev.InputDevice(key_dev)
        return True
    except Exception as e:
        print (e)
        return False
```

图 75

（2） 监听按键事件。

```
28      def listen_key_pressed(self):
29          try:
30              print ("Please press the key to test.\n")
31
32              """
33              Listen for the button to be pressed
34              event.value: 1: pressed; 0: released
35              """
36              for event in self.key_dev.read_loop():
37                  if event.type == evdev.ecodes.EV_KEY:
38                      if event.code == evdev.ecodes.KEY_PROG1 and event.value == 1:
39                          print ("User key0 pressed!\n")
40
41                      elif event.code == evdev.ecodes.KEY_PROG2 and event.value == 1:
42                          print ("User key1 pressed!\n")
43
44          except Exception as e:
45              if not KeyDevice.quit_flag:
46                  print (e)
```

图 76

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com